

Cofferdam

A Generic Engine for Architectural Rules

A Design Review and Literature Synthesis

Thomas Dickson · May 2026

Abstract

This review examines the design of `cofferdam` — a TypeScript code-quality analyser written in Rust — and situates it within four lineages of programming-tools literature: architectural conformance checking, code-as-data knowledge graphs, declarative analysis languages, and the practical lint-tool tradition. `Cofferdam`'s distinguishing feature is a single declarative artifact, `cofferdam.invariants.toml`, that encodes a project's architectural intent and is consumed by multiple checks at once. Around this artifact sit a typed run-scoped corpus, a three-phase check contract, and stable check identifiers that flow through suppression, baseline diffing, and machine-readable output.

I argue that the abstraction is a direct descendant of the Software Reflexion Models tradition opened by Murphy, Notkin, and Sullivan in 1995, with a substrate today realised as flat tables of import and export records but trending toward a queryable in-memory knowledge graph in the manner of Code Property Graphs (Yamaguchi et al., 2014) and Meta's Glean. The open architectural question is how to lift the rule layer into a generic predicate language without sacrificing the surface ergonomics that make per-rule checks easy to write. The review concludes with a `v1`→`v3` sequencing that treats the predicate DSL as the highest-leverage near-term work and the substrate promotion as the natural medium-term consequence. The genericity that would let `cofferdam` generalise beyond TypeScript — to schema-IDL, infrastructure-as-code, SQL migrations, or any other domain whose architecture can be described as a graph plus a set of rules — is shown to follow from a clean separation of two contracts: an adapter contract that translates source artifacts into the canonical graph, and a rule contract that operates over the graph without referring to language specifics.

Contents

Abstract	1
1 Introduction: the problem of architectural drift	3
2 The cofferdam abstraction	3
2.1 Four customisation layers	3
2.2 The Check trait and its three phases	4
2.3 The typed corpus	4
2.4 The invariants spec as shared truth	4
2.5 Capability declaration	5
3 The architectural-conformance lineage	5
4 Code as data: the knowledge-graph tradition	5
5 Rules as queries: the declarative-analysis tradition	6
6 The two contracts: separating logic from language	7
6.1 The adapter contract	8
6.2 The rule contract	8
6.3 The seam where genericity breaks	8
6.4 The shared-rule test	8
6.5 Three asymmetries the design must manage	8
7 The convergence: cofferdam as in-memory knowledge graph	9
7.1 What the project graph wants to be	9
7.2 The query language question	9
7.3 Sequencing	9
7.4 Risks	9
8 Open work mapped to the literature	10
9 Conclusion	10
10 References	11
10.1 Architectural conformance	11
10.2 Knowledge graphs over code	11
10.3 Declarative analysis and rule languages	11
10.4 Theoretical and historical foundations	11
10.5 The lint-tool lineage	11

1 Introduction: the problem of architectural drift

Software architecture is enforced by what the build refuses to accept. A team can document its layering, its public-API rules, its forbidden imports, its size limits and naming conventions; if none of that is checked mechanically on every commit, it drifts. The history of programming-tools research over the last three decades can be read as repeated attempts to mechanise this enforcement: from compiler warnings in the 1980s, through the structural conformance models of the 1990s, the declarative-analysis languages of the 2000s, the code-as-data systems of the 2010s, and the current generation of pragmatic linters and code-search tools.

The leverage produced by these efforts has been uneven. Per-rule configuration scales linearly: every new rule needs its own keys, its own documentation, its own per-project tuning. An architect who wants to express “code in `app/` may not import from `infra/db`” once will find themselves writing it twice or three times — once as an ESLint config, once as a CI script, perhaps a third time as a tribal-knowledge entry in a runbook — because no single artifact represents what the codebase is supposed to be. The recurring leverage idea, present across reflexion models, fitness-function literature, and modern architectural linters, is to extract architectural intent into a single declarative artifact that many rules consume.

Cofferdam is a recent realisation of that idea, narrowed to TypeScript. The tool ships approximately forty built-in checks across five categories (Consistency, Design, Readability, Refactor, Warning) inherited from Elixir’s Credo. Most of those checks are local: a function with too many parameters, a `==` rather than `===`, a line that exceeds a configured width. Such checks would not justify a review on their own; they are well-trodden territory. The interesting parts of cofferdam are cross-cutting. A single `cofferdam.invariants.toml` file declares layers, allowed inter-layer imports, frozen boundaries (subtrees marked as legacy and forbidden new code), public-API entry points, and arbitrary forbid/require import rules. Multiple checks consume the same spec, so the architecture is described once and enforced many times. An inline-comment suppression mechanism, a stable check-ID convention, and a baseline workflow let a project adopt the tool without drowning in pre-existing findings.

This review has three concerns. The first is to articulate cofferdam’s design decisions and the leverage they produce on a downstream repository, anchored to specific files in the codebase. The second is to situate those decisions in their literature context: what cofferdam inherits from Murphy, Notkin, and Sullivan’s reflexion models; from Yamaguchi’s Code Property Graphs; from Bravenboer and Smaragdakis’s Datalog-for-static-analysis tradition; from de Moor’s QL/CodeQL; and from the practical lint-tool lineage exemplified by Credo, ESLint, and Clippy. The third is to identify the architectural changes that would let cofferdam generalise beyond TypeScript: a separation of rule logic from source language, a substrate promotion from typed-slot store to queryable graph, and a predicate DSL that targets the substrate directly.

The argument the review will make can be summarised in one paragraph. Cofferdam is a typed-corpus rule engine in the lineage of reflexion models and code-property graphs. Its current substrate is two flat tables of import and export records consumed by hand-coded queries in each check’s finalize phase; the natural and well-precedented next step is to promote that substrate to a first-class queryable graph and to lift the rule layer into a predicate DSL that targets it directly. Doing so cleanly requires a separation of language-specific adaptation from domain-agnostic rule logic that the literature has worked out repeatedly across CPG, CodeQL, Glean, and Kythe. Cofferdam does not need to replicate any of these systems, but it has reached the point where ignoring them is a missed opportunity. The remainder of the review develops this argument, with the technical detail and citation needed to make it operational rather than merely suggestive.

2 The cofferdam abstraction

Cofferdam is a Rust workspace whose load-bearing crates are `cofferdam-core` (defining the `Check` trait, contexts, and metadata), `cofferdam-engine` (orchestration, parser loop, finalize ordering), `cofferdam-checks` (the built-in rule library), `cofferdam-formatters` (text, JSON, SARIF output), and `cofferdam-cli` (the user-facing binary). A separate `@cofferdam/check-sdk` npm package lets users write rules in TypeScript that the engine loads via the `plugins` array in configuration. The architectural decisions worth examining live almost entirely in `cofferdam-core`.

2.1 Four customisation layers

A downstream repository influences cofferdam’s behaviour through four nested layers, narrowest to widest. The first is the set of built-in checks compiled into the binary — the opinionated taxonomy of TypeScript smells. The second is per-check options declared in `cofferdam.toml` under tables of the form `[checks."Category.Name"]`; these tune individual rules with narrow knobs (limits, allow-lists, severity overrides) and are validated at engine startup against a static `OptionSpec` schema attached to each check. The third is the project-wide architectural spec at `cofferdam.invariants.toml`, which declares layers, allowed inter-layer imports, public-API entry points, frozen boundaries, and generic forbid/require import rules. The fourth is the plugin SDK, which loads user-authored TypeScript checks alongside the built-ins; findings produced by plugins flow through the same priority computation, suppression, baseline diffing, and output channels as native findings.

The third layer is the centre of mass. Per-rule configuration — the second layer — is the dominant pattern in linters of the ESLint/Rubocop generation, and its limitation is that architectural intent is encoded N times if N rules care about it. Cofferdam’s choice is to declare the architecture once, in a single file, and have multiple checks consume the same declaration. Today four built-in checks (`Design.LayerViolation`, `Design.OrphanExport`, `Design.BoundaryFrozen`, `Design.InvariantViolation`) read the invariants spec; the spec is also consumed by `cofferdam advise`, an output mode that emits the rules applicable to a given file before any edit is made, so an LLM agent can plan against the architecture rather than reverse-engineering it from violations. Three audiences — humans reading documentation, rule implementations doing enforcement, and agents planning edits — read the same artifact.

2.2 The Check trait and its three phases

Every rule in `cofferdam`, whether built-in or plugin, implements the `Check` trait declared in `crates/cofferdam-core/src/check.rs`. The trait has a small surface: a `meta()` method returning static metadata, a per-file `run()` method that emits findings, and two optional methods that activate phase-specific orchestration. A check declaring `consistency: true` in its meta opts into a two-pass mode: pass one collects per-file evidence (for example, the relative frequency of single versus double quotes, or the modal indentation), and pass two flags files that deviate from the discovered convention. A check that needs to see all files before it can emit findings (orphaned exports, duplicate-block detection, layer-violation analysis) implements `finalize()`, which runs once per analysis after all per-file phases have completed.

The phases are not merely organisational. Each phase has its own context type — `CheckContext` for `run` and `pass2`, `FinalizeContext` for `finalize` — and the type system constrains what a check can touch in each phase. `FinalizeContext` does not expose a current file or a parsed AST, because no such thing exists at `finalize` time; only the corpus is available. This constraint is invisible in practice (a check author writes the natural code) but it is what makes phase-level parallelism safe and what makes the orchestration legible to readers of unfamiliar checks: a method's signature tells you which phase you are in.

A more recent refinement, motivated by a real bug recorded as `cd-wqc`, is the two-phase `finalize`. A check whose meta sets `observes_findings: true` has its `finalize` call deferred until every other check's `finalize` has emitted; the engine then rebuilds a corpus snapshot of the full pre-filter finding set and runs the observers against it. Today the only observer is `Consistency.UnusedSuppression`, which flags inline-comment suppressions that no longer correspond to any emitted finding. The mechanism is general — it would support meta-checks for rule co-firing, suppression-noise audits, or finding-on-finding rules — and represents a clean way of expressing “rules about rules” using the same machinery as ordinary rules.

2.3 The typed corpus

Cross-file checks need shared state across the per-file phases. The `CorpusIndex`, defined in `crates/cofferdam-core/src/corpus.rs`, is the run-scoped store. One instance is built per analysis, passed by reference into every `CheckContext`, and handed to `FinalizeContext` after the per-file phases complete. Slots are addressed by `CorpusKey<T>` constants, where the string name and the type parameter together establish identity. Two checks share storage by referencing the same constant; mismatched types on the same name panic at runtime, an error caught immediately on first run and acceptable for compile-time-reviewed built-ins.

```
static IMPORTS: CorpusKey<Vec<ImportRecord>> =
    CorpusKey::new("cofferdam.graph.imports");

ctx.corpus.with_slot(&IMPORTS, |slot: &mut Vec<ImportRecord>| {
    slot.extend(extract_imports(file, parsed));
});
```

The locking design deserves comment because it is a subtle decision. The outer `RwLock<HashMap>` is held only for slot lookup or insertion; each slot has its own `Mutex` for value access. Two checks targeting distinct keys can therefore run their `with_slot` closures concurrently — only same-key access serialises. This is the third path between two common designs in static-analysis frameworks: a single typed scratchpad on the trait (forces every shared-state question through one God-object), and a stringly-typed `HashMap<&str, Box<dyn Any>>` (forces every reader to downcast and lose type safety). `CorpusKey<T>` is typed, scoped, and parallel-friendly.

The corpus is the substrate of cross-file analysis in `cofferdam`, and the question this review will keep returning to is what shape that substrate should ultimately take. Today it is a flat namespace of typed slots; the project's import/export graph happens to live across two well-known slots, with hand-coded queries reconstructing graph traversal in each `finalize` body. The literature on knowledge graphs over code (section 4) suggests this is a transitional shape rather than an end state.

2.4 The invariants spec as shared truth

The format of `cofferdam.invariants.toml` is intentionally narrow. The file describes layers as named glob patterns over the project tree, with a separate `[layers.allow]` table declaring which layers may import from which. Public-API entry points are listed as either filesystem paths or `package.json:<key>` pointers (the latter resolved to a concrete set at spec-load time). Boundaries are globs with associated metadata, of which `frozen = true` is the principal flag today — it means new code under the matching pattern is forbidden. Generic invariants are named rules with `forbid_imports`, `require_imports`, and `from_layers` attributes, evaluated as predicates over the import graph.

```
[layers]
infra = ["src/infra/**"]
domain = ["src/domain/**"]
app = ["src/app/**"]

[layers.allow]
domain = ["infra"]
app = ["domain", "infra"]

[invariants]
"no-direct-db-access" = {
    forbid_imports = ["src/infra/db"],
    from_layers = ["app"]
}
```

The narrowness is the design choice. A more expressive format would let projects encode arbitrary predicates, at the cost of complicating both the parser and the consumer surface; cofferdam takes the position that the simple cases (layer-and-allowlist, public-API exemption, forbid/require imports) cover the majority of real architectural rules, and that the harder cases should be the territory of a future predicate DSL rather than ad-hoc TOML extensions. Section 7 returns to this trade-off.

2.5 Capability declaration

`CheckMeta`, the static metadata returned from `meta()`, is the engine’s planning surface. A check declares its category, base priority, default severity, options schema, and several capability flags: `requires_types` (the check needs a type-aware backend), `consistency` (run me in two-pass mode), `observes_findings` (run my finalize after every other check’s), and `autofix` (I produce mechanical text edits). The engine schedules a run from declarations alone; it never has to introspect a check at runtime to decide what phase or routing it needs.

The principle is capability declaration over capability inference, and it is what keeps the engine and the check trait decoupled. New capabilities are added to `CheckMeta` and the engine learns to route them; existing checks that do not opt in are unaffected. New built-ins or plugins read like configuration: declare flags, get orchestration. The principle has cost — a flag without an engine implementation is a doc pledge with no code behind it, and at least one such flag (`requires_types`) has been declared without its router yet existing — but the cost is bounded by being explicit in the metadata rather than implicit in trait behaviour.

3 The architectural-conformance lineage

Cofferdam’s invariants spec sits in a well-developed research lineage that has mostly been carried by tooling, not academic publication. The foundational work is Murphy, Notkin, and Sullivan’s “Software Reflexion Models,” presented at FSE in 1995 and elaborated in TOSEM in 2001. The contribution was a formalism for comparing a high-level architectural model of a system against its actual source-code dependencies, with the discrepancies (“convergences,” “divergences,” and “absences”) computed mechanically. The reflexion model is the direct ancestor of cofferdam’s `[layers]` plus `Design.LayerViolation`: an architect declares the intended structure, a tool checks it against extracted facts, deviations are reported.

Murphy’s contribution was conceptual rather than syntactic. The 1995 paper does not propose a configuration format or a query language; it proposes that the comparison itself is a tractable, useful, and incrementally-applicable engineering practice. The argument has held up. Every subsequent architectural-conformance system — Sangal et al.’s Dependency Structure Matrix work (OOPSLA 2005), commercial tools like Lattix, Structure101, Sonargraph, NDepend, ArchUnit, and now cofferdam — can be read as a particular implementation strategy for the same underlying problem.

The Dependency Structure Matrix tradition is worth pausing on because it represents a different surface choice. Sangal et al. organised dependencies as a square matrix indexed by modules, with cell shading indicating dependency direction and weight; the architect’s intent was encoded as forbidden zones in the matrix (typically below the diagonal). The matrix made cyclic dependencies visually obvious in a way that text-based reports do not, and it served as the conceptual basis for several commercial tools. Cofferdam does not adopt the matrix surface, but the underlying separation — declared structure as one artifact, extracted facts as another, comparison as the report — is preserved.

Ford, Parsons, and Kua’s “Building Evolutionary Architectures” (O’Reilly, second edition 2022) coined the practitioner term *architectural fitness function*, by which they mean any mechanism that produces a continuous signal about a system’s architectural health: a test suite for layering rules, a build-time check for cyclomatic complexity, a runtime metric for service coupling. Fitness functions are an umbrella concept; cofferdam is, in their vocabulary, a *structural, atomic, triggered* fitness-function system, one that runs on commit and emits per-file findings. The Ford-Parsons-Kua book is more useful for naming the family than for prescribing a specific architecture, but the family is real and cofferdam belongs to it.

ArchUnit, the Java library, deserves separate comment because it represents a different design choice within the same lineage. Where cofferdam puts architectural intent in a configuration file, ArchUnit puts it in JUnit tests:

```
classes().that().resideInAPackage("..app..")
    .should().notDependOnClassesThat()
    .resideInAPackage("..infra.db..");
```

The trade-off is real. A fluent API in the host language gets IDE autocomplete, type-checked rule construction, and the full expressive power of Java; a configuration file gets non-Java consumers (other tools, agents, documentation pipelines), human-readable diffability, and a clean separation between rule and runtime. Cofferdam takes the configuration-file path, but the choice is not obviously dominant; the ArchUnit DSL has shipped real architectural rules in real projects for years.

Where cofferdam sits in this lineage is at the intersection of three commitments. From Murphy’s reflexion models it inherits the idea that comparison is the right abstraction. From DSM/Lattix it inherits the recognition that structural rules are valuable and addressable independent of behavioural correctness. From the fitness-function school it inherits the practical commitment to running on every commit, not as a periodic audit. What cofferdam adds, at this point in the lineage, is a small typed corpus and a three-phase check contract that lets cross-file rules be expressed cleanly, and the choice to put the architectural specification in a separate artifact from per-rule configuration — distinguishing *what is the codebase* from *how should this rule behave*.

4 Code as data: the knowledge-graph tradition

The technical question of how to represent source code for analysis has produced a separate but converging tradition. Where the reflexion-models lineage focused on the comparison itself, the code-as-data tradition focused on the substrate: how do you store and query the facts about source that a rule needs to see?

The pivotal paper is Yamaguchi, Golde, Arzt, and Rieck’s “Modeling and Discovering Vulnerabilities with Code Property Graphs,” presented at IEEE Security and Privacy in 2014. The Code Property Graph (CPG) merges three previously-distinct representations — the abstract syntax tree, the control-flow graph, and the program dependence graph — into a single property-graph data structure. Nodes are syntactic constructs; edges encode AST-child, control-flow, or data-flow relationships, depending on type. A vulnerability pattern is then a graph query: find a node with a particular AST shape that has a data-flow edge from an untrusted-input source. The key insight is not the graph data structure (graphs over code predate the paper by decades) but the unification: a single representation supports the queries that previously required walking three different structures.

Yamaguchi’s Joern (joern.io) is the open-source implementation. A user analyses a codebase by extracting it into a CPG, then querying with a Scala-based DSL or a Python wrapper. The query language is graph-traversal-flavoured, and the substrate is explicit: one writes `cpg.method.name("authenticate").parameter` rather than reaching for parser-specific node types. The system is generic across languages because the CPG schema is language-neutral (with language-specific extensions); the same query can in principle work for C, Java, Python, or JavaScript code if their respective frontends emit conformant CPGs.

Google’s Kythe (kythe.io) is contemporary with the early CPG work but pursues a different goal: cross-language semantic indexing for IDE-style queries. The Kythe schema describes nodes (anchors, semantic tokens, files) and edges (defines, refers to, completes) at a granularity tuned for cross-reference and type-resolution queries rather than vulnerability detection. The interesting design contribution is that the schema is the interface between language-specific extractors and language-agnostic consumers; an indexer for a new language is correct by construction if it emits the right node and edge types. This separation of frontend and consumer is the analogue, in industrial code-indexing, of the language-versus-logic separation argued for in section 6.

Meta’s Glean (glean.software) is the most recent and arguably the most ambitious system in this tradition. Open-sourced in 2021 and described in a 2024 Meta engineering post, Glean is a fact-based code-indexing system used in production at Meta to power code search, IDE navigation, and analytical tooling. Facts are organised by schema; schemas are versioned; queries are written in Angle, a Datalog-flavoured query language. What distinguishes Glean from CPG and Kythe is the production focus on incremental ingestion, multi-language federation, and durable storage at organisation scale. A naive query against tens of billions of facts is intractable; Glean’s investment is in indexing strategies, query planning, and incremental fact derivation. The design space here is rich and has not been fully published; the 2024 engineering post is the best public account.

The Sourcegraph LSIF and SCIP formats represent a different kind of move: standardising the schema. Where CPG, Kythe, and Glean each define their own node and edge taxonomies, LSIF (Language Server Index Format) and its successor SCIP (Source Code Index Format) are formats that any indexer can emit and any consumer can read. The standardisation is incomplete — LSIF/SCIP target IDE-style queries (definitions, references, hover) rather than the broader analytic queries Glean supports — but the ambition matters: a community schema means a code-graph file extracted by tool A can be consumed by tool B, even across languages.

Cofferdam’s position relative to this tradition is partial. The project’s import and export graph already lives in the corpus, in two well-known slots defined in `crates/cofferdam-core/src/graph.rs`:

```
pub static IMPORTS: CorpusKey<Vec<ImportRecord>> = ...;
pub static EXPORTS: CorpusKey<Vec<ExportRecord>> = ...;
```

`ImportRecord` carries `from_file`, `source_specifier`, `resolved` (the absolute path of the imported module when resolution succeeded), and a list of imported names with use-counts. `ExportRecord` carries the file and the exported name. The graph is implicit in these tables: nodes are files and symbols; edges are the import-specifier relationships. Cofferdam’s graph-aware checks (`Design.OrphanExport`, `Design.LayerViolation`, `Design.ImportCycle`, `Design.DeadExport`) reconstruct queries by joining these tables in their `finalize` bodies.

The shape is functional but, by the standards of CPG/Kythe/Glean, primitive. A query like “*find symbols exported from layer X that are reachable from layer Y by a chain of import edges of length at most three*” is awkward to express against `Vec<ImportRecord>`; it is natural against a graph store with transitive-closure semantics. Cofferdam is not yet at the scale where ad-hoc joins become a performance problem (typical TS projects analysed today are in the low thousands of files), but the expressiveness ceiling is a design-time issue independent of performance. A predicate DSL ambitious enough to encode meaningful architectural rules will outgrow flat tables before it outgrows in-memory size.

The forward direction this tradition implies for cofferdam is to promote the project graph from “two well-known slots in the corpus” to a first-class queryable subsystem. Not necessarily Glean-scale or with Kythe’s cross-language ambitions; an in-memory store with Datalog- or Cypher-flavoured query semantics, indexed for transitive closure and edge-typed traversal, would be sufficient for current and foreseeable use. The substrate is roughly right; what is missing is the query layer and the abstraction over the storage tables.

5 Rules as queries: the declarative-analysis tradition

The complement to the code-as-data tradition is the tradition of expressing rules as queries rather than as imperative code. The two are independent in principle (one can store code as data and still write rules as Java methods, or store code as ASTs and write rules in Datalog) but they evolve together because each magnifies the other.

Bravenboer and Smaragdakis’s “Strictly Declarative Specification of Sophisticated Points-to Analyses” (OOPSLA 2009) is the canonical demonstration of how far the declarative-analysis idea can be pushed. The paper presents Doop, a framework for whole-program points-to analysis in which the entire analysis — from intermediate-representation extraction through context-sensitive pointer resolution — is specified in Datalog. The Datalog evaluator (originally LogicBlox, now typically Soufflé) executes the specification against the extracted facts; the analysis writer never imperatively walks an AST. The result was both more expressive than then-current points-to analyses and faster, because the Datalog engine could optimise rule evaluation in ways a hand-coded analysis could not.

The lessons from Doop for cofferdam are several. First, the Datalog/declarative tradition offers a credible scaling path: a substantial body of static-analysis work is expressible as logic rules over relations, and the evaluation strategies (semi-naïve evaluation, magic sets, incremental update) are well-developed. Second, there is a real gap between the analysis writer’s mental model and the substrate: Doop users write rules, not loops, and the Datalog engine makes them tractable. Third, the substrate-as-relations model is naturally extensible; adding a new fact type (a new kind of edge, a new analysis input) does not require changing the rule language.

Avgustinov, de Moor, Peyton Jones, and Schäfer’s “QL: Object-oriented Queries on Relational Data” (ECOOP 2016) describes the language behind Semmlé’s CodeQL system (now owned by GitHub). QL adds object-oriented features to Datalog: classes that are subsets of a base relation, methods that produce derived relations, polymorphic dispatch over classes. The language compiles to standard Datalog and runs on a relational database. The contribution is ergonomic rather than expressive — everything one writes in QL one could write in plain Datalog — but the ergonomics matter: a CodeQL query for a security vulnerability reads more like Java code than like Prolog, and the audience that can write one is correspondingly broader.

CodeQL itself is the most operationally successful of the declarative-analysis systems; it ships with most of GitHub’s security-scanning offering and has a substantial library of community-contributed queries. The pricing model is enterprise, but the language and a substantial fraction of the standard library are open. For a cofferdam-future considering a predicate DSL, the QL paper is essential reading on the question of “how should a rule language feel”; the answer it gives — like a small object-oriented language whose objects happen to be relations — is one credible point in the design space.

The Coccinelle work, Padioleau, Lawall, Muller, and Hansen’s “Documenting and Automating Collateral Evolutions in Linux Device Drivers” (EuroSys 2008), occupies a different niche. Where Doop and QL are query-oriented (find findings), Coccinelle is rewrite-oriented (find and transform). A Coccinelle “semantic patch” specifies a syntactic pattern with metavariables and a transformation; the engine matches the pattern across a codebase and applies the transformation. The contribution to the present discussion is the demonstration that declarative pattern languages can be ergonomic for non-trivial code work; semantic patches are not Datalog, but they are not imperative either. Cofferdam’s autofix surface — a check returns a `TextEdit` for a finding — is currently imperative; a future direction in which fixes are also declarative patterns is plausible and well-precedented.

Semgrep (semgrep.dev) is the most pragmatic of the modern descendants. The Semgrep rule format is YAML-based, with patterns that look like the language being analysed but with `$X` metavariables that match arbitrary subterms:

```
rules:
- id: no-eval
  pattern: eval(...)
  message: Avoid eval; use a parser
  severity: WARNING
  languages: [javascript, typescript]
```

The expressive ceiling is much lower than Doop or CodeQL — there is no transitive closure, no cross-function dataflow without paid extensions, no rule reuse beyond pattern-includes — but the surface is approachable. A developer who knows the source language can write a Semgrep rule in minutes; the same developer writing CodeQL takes substantially longer. This is the lower bound on rule-DSL ergonomics that any cofferdam predicate DSL will be measured against; the appeal of “I can read this and roughly write one” is high.

The theoretical floor is Cousot and Cousot’s “Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints” (POPL 1977), one of the most-cited papers in the static-analysis canon. The contribution is a general framework for sound static analyses based on lattices and fixpoint computation: an abstraction function from concrete to abstract states, a transfer function for each program statement, a fixpoint operator that yields the analysis result. Cofferdam does not claim soundness in the abstract-interpretation sense; its checks are heuristic, optimised for false-negative tolerance over false-positive tolerance, and the gap between cofferdam-style checks and abstract-interpretation-style soundness is large. But if cofferdam ever extends toward verified architectural rules — where *no* violation is missed, not just *few* — abstract interpretation is the framework. The Cousots’ paper is the right reference, and several modern textbooks (Nielson, Nielson, and Hankin’s “Principles of Program Analysis,” for example) develop the machinery in implementable form.

The surface-design question for a cofferdam predicate DSL is bounded by these references. A Datalog-flavoured language (in the Doop/QL lineage) maximises expressive power and substrate-level optimisation, at the cost of unfamiliarity to most developers. A pattern-match language (in the Semgrep lineage) maximises ergonomics, at the cost of expressive ceiling. A graph-query language (in the Joern/Glean Angle lineage) splits the difference: ergonomic to read for graph-shaped predicates, awkward for the per-file-shape predicates that dominate practical lint catalogues. The answer is probably hybrid: a base substrate of graph queries with sugar for common per-file shapes, in the way that QL adds sugar over Datalog without abandoning Datalog semantics.

6 The two contracts: separating logic from language

The synthesis of the previous two sections is that cofferdam is converging on a familiar pattern: a typed graph substrate populated by language-specific extractors, queried by language-agnostic rules. The convergence is desirable not just for performance or expressiveness but for genericity — the property that the engine can serve more than one source language without forking. Genericity is not a virtue in the abstract; it is a virtue insofar as a single rule, written once, can apply to multiple domains, and a single engine, maintained once, can support multiple adapters. Cofferdam is TypeScript-only today; whether it remains so is partly a question of strategic focus and partly a question of architectural choice.

The architectural question — what would it take for cofferdam to support, say, schema-IDL or infrastructure-as-code in addition to TypeScript — has a clean answer in the literature: separate the system into two contracts that cannot leak into each other.

6.1 The adapter contract

The first contract is the adapter contract. An adapter is the only place language- or format-specific code lives. It takes source artifacts as input — TypeScript files, SQL migration scripts, Terraform manifests, GraphQL schemas, whatever the domain consists of — and produces typed nodes and edges in the canonical graph as output. An adapter is allowed to extend the schema with domain-specific node and edge types: `sql.column`, `iac.resource`, `gql.field`, `ts.symbol`. These extensions are declared upfront and registered with the engine, so consumers (rules) can target them by stable name.

What an adapter is forbidden from doing is the load-bearing part of the contract. An adapter must not see rules. An adapter must not know about findings. An adapter must not call user code or interact with configuration in ways that would couple it to a specific rule set. Its job is graph population and nothing else. A failure to enforce this discipline produces an adapter that “knows” about specific rules and emits domain-specific facts only when those rules are enabled, which is an observable but subtle form of coupling that becomes intractable at scale. Glean, Kythe, and CPG all enforce this separation by construction — the indexer is a separate executable from the consumer, and the schema is the interface.

6.2 The rule contract

The second contract is the rule contract. A rule is the only place logic lives. It takes graph queries as input, optionally a file handle if it must dip into source for data not promoted to the graph, and produces `Issue`s as output. A rule may use domain-specific predicates exposed by adapters — a rule about SQL migrations can query `sql.column` nodes — but it cannot reach into the source-language AST that the adapter consumed. The AST is the adapter’s private state; the graph is the adapter’s public output.

The rule contract is what makes the rule layer language-agnostic. A rule does not know whether it is operating on TypeScript or SQL; it knows only that it is operating on nodes and edges in the canonical graph plus any registered domain extensions. The same rule, in principle, can apply to multiple domains if the domains share enough graph structure; this is the test of genuine genericity discussed in the next subsection.

6.3 The seam where genericity breaks

The temptation that breaks the model is rule authors peeking at AST shapes. “*Just give me the `ImportDeclaration` node, I want to check its `assertions` clause.*” It is small, it is convenient, it is exactly the kind of escape hatch that a sympathetic API designer is tempted to provide. It is also where domain-specific knowledge leaks back into the logic layer, because the rule that consumes an `ImportDeclaration` node is no longer language-agnostic: it knows about TypeScript ASTs.

The defence the literature consistently lands on is that anything a rule needs from the source must be expressible as a graph node or edge attribute. If the attribute is missing, that is a missing schema element to add to the canonical layer — and to be made available to every other rule via the adapter — not a hole in the rule API to be patched with an AST escape hatch. The discipline is the same one that keeps a database engine queryable: consumers query, they do not walk the storage format. Cofferdam’s storage format ought to be the canonical graph; the rule layer’s only legal API ought to be queries against it.

This is, importantly, an architectural rule for cofferdam-the-tool to enforce on its own contributors and plugin authors — a constraint on the engine API, expressible as documentation and as type signatures, and one that ought to be stable across the engine’s lifetime.

6.4 The shared-rule test

A practical test for whether the separation has been achieved is whether two parallel domains can share a rule. “*No public-internet exposure from a `prod-*` module*” should be one rule that applies to both Terraform and Pulumi: both adapters produce `iac.resource` nodes; both attach a `public_ip` attribute when the resource is publicly addressable; the rule queries on resource type and attribute and emits a finding. If the rule is one rule, the separation is real. If it must be written as `terraform_no_public_ip` and `pulumi_no_public_ip`, the engine is a plugin system, not a generic substrate.

The test extrapolates. “*All public exports must originate from a documented entry point*” should apply to both TypeScript modules and GraphQL schemas if both adapters produce `export` edges and the spec declares `entry_point` nodes. A naming-convention rule, “*no resources may be named with a verb-prefix*”, should apply to AWS Lambda function names and GraphQL field names alike if both are represented as `name`-attributed nodes. The rule cares only about graph shape; the adapter cares only about translating its source into that shape. When this works, the engine has achieved the layered separation. When it does not, the layering is at best aspirational.

6.5 Three asymmetries the design must manage

Three honest asymmetries complicate the layered ideal. The first is `Span`: today byte-offset based, fine for text-source artifacts, awkward for binary or generated artifacts. Generalising to a `Location { uri, range }` type that supports more than byte ranges is cheap to do up-front and expensive to do later, because span flows through `Issue`, `RelatedSpan`, suppression directives, baseline files, and SARIF output. The second is identity: each domain needs a stable-ID story for findings, because suppression directives, baselines, and incremental analysis all depend on stable identity across runs. Today identity is `(check_id, file, span_hash)`; SQL migrations would prefer `(rule, migration_file, statement_index)`; schema-IDL would prefer `(rule, schema_file, type_name)`. The engine has to ask the adapter for an identity scheme rather than assuming text-span hashes will work.

The third is taxonomy. The five categories that cofferdam inherited from Credo are tuned for code-quality findings on object-functional source. A schema validator wants `Breaking | NonBreaking | Convention`. A SQL-migration domain wants `Reversible | Irreversible | DataLoss`. The configurable-taxonomy work that cofferdam’s documentation has pledged but not implemented (`crates/cofferdam-core/src/check.rs:6` claims projects can *add* categories, but the `Category` enum at line 29 is closed) is therefore not a doc fix; it is a prerequisite for non-TS domains. Adapters need to register their own taxonomies;

the engine needs to display them coherently in reports; configuration has to accept domain-aware severity overrides. None of this is hard; all of it is necessary.

7 The convergence: cofferdam as in-memory knowledge graph

The synthesis of sections 4, 5, and 6 implies a forward direction for cofferdam that is more specific than “improve incrementally.” The corpus today is a flat namespace of typed slots; the project graph lives implicitly across two of those slots; rule logic reconstructs graph queries by hand. The consistent direction the literature points toward is to make the project graph first-class: a queryable substrate that rules target directly, with the existing slot store retained for the cases where typed scratchpad semantics are genuinely what is wanted (a check accumulating per-file evidence for its own pass-2 use, for instance).

7.1 What the project graph wants to be

The graph cofferdam wants is not a heavyweight system. It does not need Glean’s billions-of-facts indexing infrastructure or Kythe’s cross-language schema. It needs to support, for typical TypeScript projects in the low thousands to low tens of thousands of files: per-file ingestion of nodes and edges produced by the adapter; transitive-closure queries (reachability, distance, path existence) over edges; edge-typed traversal (imports-as-type vs imports-as-value, exports-as-default vs exports-as-named); attribute filtering (node where attribute matches); and incremental update on file change (drop facts contributed by the changed file, ingest new facts).

In-memory implementations of these primitives are well-trodden. A naive in-memory triple store with hash-indexed access patterns is a few hundred lines of Rust. A more sophisticated implementation with B-tree indexes, query planning, and stratified-negation Datalog evaluation is a substantially larger investment, but achievable, and several open-source Datalog engines (Crepe, Ascent, the Soufflé library) provide most of it. The choice between writing from scratch and embedding an existing engine is not the focus of this review; both are credible.

7.2 The query language question

The substrate question is independent of the surface-language question, and both must be answered. Section 5 discussed three distinct surface choices: Datalog-flavoured (maximal expressiveness, least familiar), graph-traversal-flavoured (ergonomic for graph queries, awkward for per-file-shape), pattern-match-flavoured (maximal accessibility, lowest expressive ceiling). The trade-offs are real and not resolvable by reasoning about surface alone.

The pragmatic answer the recent literature converges on is hybrid: a substrate based on graph or relational semantics, with a surface that includes both general-purpose query syntax and domain-specific sugar for common predicates. CodeQL works this way — QL is Datalog plus object-oriented sugar; Glean’s Angle is Datalog with schema-aware extensions; Joern’s Scala-based query DSL is graph-traversal with case-class-based pattern matching. None of these is “Datalog” in the bare sense; none is “Cypher” or “Semgrep” either. They are designed surfaces over a common substrate.

For cofferdam, the implication is that the predicate DSL designed for `cd-9hp.1` should not commit to a single surface family. A v1 surface that handles per-file-shape predicates ergonomically (in the Semgrep idiom) and provides an escape hatch for graph queries (in the QL or Cypher idiom) covers the space; a v2 substrate change can land underneath without breaking v1 rule files, provided v1 was designed with the graph backend in mind.

7.3 Sequencing

A credible sequencing of the work is as follows. v1 is the predicate DSL over the existing flat-corpus shape: a `[invariants.scripted]` block in `cofferdam.invariants.toml`, a small embedded scripting layer (Rhai, Starlark, or a custom mini-language), and a single Rust check that interprets the table. The v1 surface provides perhaps eighty percent of the expressiveness of a graph-backed system, at a fraction of the engineering cost. It is designed with v2 in mind — the syntax allows for transitive-closure predicates, edge-typed traversal, and domain-extension namespaces, even if v1 does not implement all of them. v2 promotes the project graph to first-class. The flat `IMPORTS / EXPORTS` slots remain populated for backward compatibility with built-in checks; a new graph subsystem ingests the same data into a queryable form; the v1 DSL surface is lifted to compile against the graph instead of against the tables. v2 is the engineering project that makes transitive-closure queries tractable at scale.

v3 is the adapter contract. A non-TypeScript adapter (perhaps SQL migrations, perhaps GraphQL schemas) is wired through the engine and produces graph facts under a domain-extension namespace. The v1 DSL surface absorbs the new domain because adapters extend the schema, not the rule language. The taxonomy work (`cd-9hp.3`) and the span/identity refactoring become non-optional at this stage.

Each step is independently shippable; each step preserves the value of the previous step’s work; none of them is a multi-month rewrite. This is approximately what the open-work bead set in `cd-9hp` already encodes, though the beads are written at the level of individual features rather than as a unified sequence.

7.4 Risks

Three risks deserve naming. The first is premature schema commitment. The canonical graph schema, once published and consumed by rules and external tools, is hard to evolve incompatibly. The Glean and Kythe schemas underwent significant change over years; both projects retain back-compatibility burdens. Cofferdam’s response should be schema versioning from day one, with adapters and rules declaring the schema version they target.

The second is DX regression. A power user can express a rule in fewer lines of Datalog than in CodeQL; the same user can express it in fewer lines of CodeQL than in Semgrep; but the audience for Datalog is much smaller than the audience for Semgrep. If the predicate DSL is too powerful, the community of rule-writers shrinks. The mitigation is layered surface — the simple cases stay simple, the hard cases become possible.

The third is performance. Graph-query optimisation is non-trivial; a naive in-memory evaluator on a moderately-large project graph can perform poorly without indexes. Cofferdam is unlikely to need Glean-scale optimisation, but it will need to think about query performance before it ships transitive-closure predicates. The literature on incremental Datalog evaluation (Mcsherry et al.’s differential dataflow, the Soufflé incremental work) is the right starting point.

8 Open work mapped to the literature

The architecture-extension epic recorded in the beads system as `cd-9hp` enumerates eight open items. Each is a real, locally-tractable engineering project; the value of placing them in a literature context is that the priorities become legible. In approximately decreasing order of leverage:

The predicate DSL (`cd-9hp.1`) is the highest-leverage item. The literature on declarative analysis languages has worked the design space repeatedly — Datalog, Doop’s experience, QL’s sugar, Semgrep’s approachability, Coccinelle’s patterns — and the principal contribution cofferdam needs to make is *selection* from that design space rather than original research. The lever is high because every existing built-in cross-file check is, in effect, a hand-coded query that the DSL would replace; every architectural rule a project wants to add is a DSL invocation rather than a Rust patch. The DSL is also where the language-versus-logic separation gets enforced — the existence of the surface is what creates the disciplinary gradient against AST escape hatches.

Spec-contract integration tests (`cd-9hp.8`) are the second-highest-leverage item. The invariants spec is a contract; absent end-to-end tests pinning “this spec on this fixture produces these findings,” refactors of the engine’s interpretation can silently change downstream behaviour. The literature analogue is property-based testing of analysis tools — Pădurariu’s “Comparative Empirical Evaluation of Static Analysis Tools” (ASE 2018) and similar surveys — which consistently finds that reference-fixture suites are the best practical defence against semantic drift. The work is unglamorous and pays off cumulatively.

The configurable-taxonomy work (`cd-9hp.3`) is structurally a prerequisite for any non-TS domain, as discussed in section 6. Today it appears low-leverage because the project is TypeScript-only; it becomes high-leverage as soon as a second domain is on the table. The right time to do it is before the first non-TS adapter, not after.

The ts-morph worker pool (`cd-9hp.2`) unblocks type-aware checks. The relevant literature is the Doop tradition of program analysis using compiler-grade frontend information; the implementation question is whether to spawn ts-morph workers or to invest in a Rust-native TypeScript type checker (a substantial undertaking). The pragmatic choice is workers; the long-term choice depends on how much type-aware analysis cofferdam wants to do.

Incremental analysis (`cd-9hp.4`) is the daemon-mode and LSP enabler. The literature is rich — McSherry et al. on differential dataflow, the Salsa framework, Rust-analyzer’s own incrementalism — and the engineering is substantial. The project depends on the substrate decisions for v2 of the DSL: a graph store with incremental update semantics is much easier to make incremental at the analysis layer than a recomputed-each-run flat table.

The plugin corpus access work (`cd-9hp.6`) and the corpus error-handling refactor (`cd-9hp.7`) are coupled. Today the corpus’s `TypeId-mismatch` panic is a logic-error guard appropriate for compile-time-reviewed built-ins and hostile to plugin authors; the namespace-by-check-ID and fallible-API changes turn it into a usable plugin-author surface. Together they are a few weeks of work; their value depends on how much cross-file plugin work the project ends up with.

The `observes_findings` direction (`cd-9hp.5`) is the lowest-priority item but worth a deliberate decision. Today the mechanism is real but exercised by exactly one check; either it is a generic extension point that should be exercised by additional meta-checks, or it is a special-case for suppression-staleness that should be inlined into the engine. The decision is small. Postponing it costs little, but it is an unresolved design question that occasionally bites readers.

The pattern across these items is consistent: the work cofferdam needs to do is well-precedented in the literature, the engineering cost is bounded, and the value per item is increasing with the project’s ambition. Cofferdam at TypeScript-only-and-static is well-served by approximately its current design; cofferdam targeting a generic substrate-and-rule-DSL needs the items in the order described.

9 Conclusion

Cofferdam is a good citizen of three converging traditions. Its approach to architectural conformance (declared structure as a separate artifact, extracted facts compared against it, deviations reported) descends directly from Murphy, Notkin, and Sullivan’s reflexion models, refined by thirty years of subsequent tool work and arrived at independently in the contemporary practitioner literature on architectural fitness functions. Its representation of code as a typed corpus with implicit graph structure is a transitional shape, more primitive than the Code Property Graphs of Yamaguchi, the cross-language fact bases of Kythe, or the production knowledge-graphs-over-code of Meta’s Glean, but pointing in the same direction. Its rule layer is currently imperative Rust, awaiting a predicate DSL whose design space the Datalog-and-CodeQL-and-Semgrep tradition has thoroughly explored.

The argument for cofferdam to lift its substrate to a queryable graph and its rule layer to a predicate DSL is not that it is required for the current TypeScript-focused product, but that doing so unlocks generality cleanly. The two-contracts model from section 6 — adapter contract for source-language translation, rule contract for domain-agnostic logic, with the canonical graph as the only sanctioned interface between them — is the architectural choice that lets cofferdam serve more than one domain without forking. The shared-rule test is the practical criterion: when a rule about public-internet exposure can be written once and apply to both Terraform and Pulumi, the layering has been achieved.

A reader inheriting this review and the codebase would not be wrong to read it as a long argument for `cd-9hp.1` — the predicate DSL — as the highest-leverage near-term work. That would be a fair summary, but the deeper claim is broader: the predicate DSL is the lever because the substrate, the rule layer, and the genericity story all hinge on it. Implementing it well (with v2 and v3 in mind, against a substrate that can be promoted, with a surface that does not commit to a single design family) sets up a decade of capability. Implementing it poorly — as a stringly-typed configuration extension that does not survive the move to a graph backend — consumes the lever without producing the capability.

The references that follow are the suggested reading list. They are organised by the four traditions and ranked, within each tradition, by relevance to the design choices ahead of cofferdam. Most are open-access PDFs at author or institutional URLs; the two commercial titles are noted. A reader with a few weeks of evening reading time can absorb the relevant subset and arrive at the design choices for `cd-9hp.1` substantially better-armed than starting from blank.

10 References

10.1 Architectural conformance

Murphy, G. C., Notkin, D., and Sullivan, K. J. (1995). “Software Reflexion Models: Bridging the Gap Between Source and High-Level Models.” *Proceedings of the 3rd ACM SIGSOFT Symposium on Foundations of Software Engineering (FSE ‘95)*, pp. 18–28. PDF: cs.ubc.ca/~murphy/papers/rm/reflexion_model_fse95.pdf

Sangal, N., Jordan, E., Sinha, V., and Jackson, D. (2005). “Using Dependency Models to Manage Complex Software Architecture.” *Proceedings of OOPSLA 2005*, pp. 167–176. PDF: groups.csail.mit.edu/sdg/pubs/2005/oopsla05-dsm.pdf

Ford, N., Parsons, R., and Kua, P. (2022). *Building Evolutionary Architectures*, 2nd edition. O’Reilly.

ArchUnit (Java library): archunit.org

10.2 Knowledge graphs over code

Yamaguchi, F., Golde, N., Arzt, D., and Rieck, K. (2014). “Modeling and Discovering Vulnerabilities with Code Property Graphs.” *Proceedings of the IEEE Symposium on Security and Privacy*, pp. 590–604. PDF: comsecuris.com/papers/06956589.pdf

Joern (open-source CPG implementation): joern.io

Marlow, S. (2024). “Indexing Code at Scale with Glean.” Meta Engineering blog, December 2024. URL: engineering.fb.com/2024/12/19/developer-tools/glean-open-source-code-indexing. Project: glean.software

Kythe project: kythe.io

10.3 Declarative analysis and rule languages

Bravenboer, M., and Smaragdakis, Y. (2009). “Strictly Declarative Specification of Sophisticated Points-to Analyses.” *Proceedings of OOPSLA 2009*, pp. 243–262. PDF: courses.cs.washington.edu/courses/cse503/10wi/readings/p243-bravenboer.pdf

Avgustinov, P., de Moor, O., Peyton Jones, M., and Schäfer, M. (2016). “QL: Object-oriented Queries on Relational Data.” *30th European Conference on Object-Oriented Programming (ECOOP 2016)*, LIPIcs vol. 56, pp. 2:1–2:25. PDF: drops.dagstuhl.de/.../LIPIcs.ECOOP.2016.2.pdf

Padioleau, Y., Lawall, J., Hansen, R. R., and Muller, G. (2008). “Documenting and Automating Collateral Evolutions in Linux Device Drivers.” *Proceedings of EuroSys 2008*, pp. 247–260. PDF: who.paris.inria.fr/Julia.Lawall/eurosys08.pdf

Semgrep documentation: semgrep.dev

10.4 Theoretical and historical foundations

Cousot, P., and Cousot, R. (1977). “Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints.” *Conference Record of the 4th ACM Symposium on Principles of Programming Languages (POPL ‘77)*, pp. 238–252. PDF: di.ens.fr/~cousot/publications.www/CousotCousot-POPL-77-ACM-p238-252-1977.pdf

Maier, D., Tekle, K. T., and Warren, D. S. (2018). *Datalog and Logic Databases*. Morgan & Claypool.

10.5 The lint-tool lineage

Credo (Elixir): github.com/rrrene/credo

ESLint custom-rule documentation: eslint.org/docs/latest/extend/custom-rules

Clippy (Rust lints): part of the Rust standard distribution.

*Document compiled with Typst 0.14. Cofferdam architecture as of May 2026.
Open work tracked under bead epic `cd-9hp` in the project’s beads database.*